INSTITUTO SUPERIOR POLITECNICO DE TETE

Divisão de Engenharia

Licenciatura em Engenharia Informática

ENGENHARIA DE SOFTWARE II

SISTEMA DE VENDA DE BILHETES DE AUTOCARRO

Nomes	**Docente**
▪ Elton Raimundo Jemusse	**Msc**. Luís Nhacanhaca
▪ Fernando Nafiquirique Cesar Fernando	
▪ Hélder Manuel Binala	
▪ Ico Anderson Alexandre Tiago	
▪ Roberto Dias Vicente Chitiche	

Tete, Agosto de 2026

Licenciatura em Engenharia Informática

ENGENHARIA DE SOFTWARE II

SISTEMA DE VENDA DE BILHETES DE AUTOCARRO

Nomes

- Elton Raimundo Jemusse
- Fernando Nafiquirique Cesar Fernando
- Hélder Manuel Binala
- Ico Anderson Alexandre Tiago
- Roberto Dias Vicente Chitiche

Trabalho de Pesquisa apresentado ao Curso de Engenharia Informatica no ISPT, como requisito parcial para a avaliação da disciplina de **Engenhara de software II.**

_____

Docente: Msc. Luís Nhacanhaca

Tete, Agosto de 2026

Resumo

O presente trabalho aborda o desenvolvimento de um **Sistema de Venda de Bilhetes de Autocarro Interprovincial**, criado para automatizar e facilitar o processo de consulta de viagens, selecção de destinos, compra de bilhetes, pagamento, atribuição de assentos e emissão de bilhetes electrónicos. O sistema procura responder às limitações dos processos tradicionais de venda de bilhetes, proporcionando maior organização, eficiência, segurança e facilidade de acesso às informações.

O projecto aplica conceitos de **Engenharia de Software, Arquitectura de Software, princípios SOLID e Design Patterns**, visando desenvolver uma solução modular, reutilizável, manutenível e preparada para futuras alterações. São analisados os padrões criacionais Singleton, Factory Method, Builder, Abstract Factory e Prototype, com maior destaque para o **Factory Method**, utilizado na criação de diferentes métodos de pagamento, e o **Builder**, aplicado na construção do bilhete electrónico.
A arquitectura proposta utiliza **React e JavaScript no frontend, Node.js/Express no backend e MySQL na base de dados**, organizando o sistema em componentes e camadas com responsabilidades distintas. Também são utilizados diagramas UML para representar a estrutura, o comportamento e a implantação do sistema.

Por fim, são definidos testes unitários e de integração para avaliar o funcionamento do sistema e a aplicação dos princípios **SOLID** e dos padrões seleccionados. Conclui-se que a utilização adequada destes princípios e padrões contribui para uma solução mais organizada, modular, flexível e preparada para futuras evoluções.

Palavras-chave: Engenharia de Software; Sistema de Bilhetes; Arquitectura de Software; SOLID; Design Patterns; Factory Method; Builder.

Lista de figuras

Figura 1:Diagrama de Classes 17
Figura 2: Diagrama de Componentes 18
Figura 3: Diagrama de Sequência 19
Figura 4: Diagrama de Actividades 19
Figura 5: Diagrama de Estados 20
Figura 6: Diagrama de Comunicação 20
Figura 7: Diagrama de Interacção 21
Figura 8: Diagrama de Implantação 21
Figura 9: Singleton 25
Figura 10: Factory Method 36
Figura 11: Integração React + Backend 37
Figura 12: Base de dados 38
Figura 13: Singleton 38
Figura 14: Avaliação dos padrões 41

Índice

Resumo III
Lista de figuras IV
CAPÍTULO I — INTRODUÇÃO 7
1.1 Contextualização 7
1.2 Problematização 8
1.3 Pergunta de pesquisa 8
1.4 Objectivos 8
1.4.1 Objectivo geral 8
1.4.2 Objectivos específicos 8
1.5 Justificativa 9
1.6 Metodologia 10
CAPÍTULO II — FUNDAMENTAÇÃO TEÓRICA 11
2.1 Engenharia de Software 11
2.2 Arquitectura de Software 11
2.3 Princípios de Design 11
2.4 SOLID 11
2.5 Design Patterns 12
2.6 Padrões Criacionais 12
2.7 Padrões Estruturais 12
2.8 Padrões Comportamentais 13
CAPÍTULO III — ANÁLISE DO SISTEMA 14
3.1 Descrição 14
3.2 Problema 14
3. 3 Requisitos funcionais 14
3.4 Requisitos não funcionais 15
3.5 Actores 15
3.6 Casos de uso 16
CAPÍTULO IV — MODELAGEM DO SISTEMA 17
4.1 Diagramas Estruturais 17
4.1.1 Diagrama de Classes 17
4.2 Diagramas Comportamentais 19
4.2.1 Diagrama de Sequência 19
4.2.2 Diagrama de Actividades 19
4.2.3 Diagrama de Estados 20
CAPÍTULO V — APLICAÇÃO DOS PRINCÍPIOS SOLID 22
5.1 SRP — Single Responsibility Principle 22
5.2 OCP — Open/Closed Principle 23
5.3 LSP — Liskov Substitution Principle 23
5.4 ISP — Interface Segregation Principle 24
5.5 DIP — Dependency Inversion Principle 24
CAPÍTULO VI — PADRÕES CRIACIONAIS 25
6.1 Singleton 25

6.2 Factory Method 26
6.3 Builder 27
6.4 Abstract Factory 28
6.5 Prototype 28
6.6 Análise comparativa 29
6.7 Selecção dos padrões 29
6.8 Justificação da selecção 30
CAPÍTULO IX — ARQUITECTURA FINAL 31
9.1 Arquitectura proposta 31
9.2 Relação SOLID + Patterns 32
9.4 Fluxo de comunicação 33
9.5 Decisões arquitecturais 34
CAPÍTULO X — IMPLEMENTAÇÃO 35
10.1 Tecnologias 35
10.2 Estrutura do projecto 35
10.3 Implementação 36
10.3.1 React 36
10.3.2 Factory Method 36
10.4 Builder 37
10.5 Integração React + Backend 37
10.6 Base de dados 38
CAPÍTULO XI — TESTES E AVALIAÇÃO 40
11.1 Testes unitários 40
11.2 Testes de integração 40
11.3 Avaliação SOLID 41
11.5 Resultados 42
CAPÍTULO XII — CONCLUSÃO 43
REFERÊNCIAS BIBLIOGRÁFICAS 44

CAPÍTULO I — INTRODUÇÃO

1.1 Contextualização

A Engenharia de Software desempenha um papel fundamental no desenvolvimento de sistemas informáticos capazes de responder às necessidades de organizações e utilizadores de forma eficiente, segura e sustentável. Com o crescimento da utilização de tecnologias digitais, torna-se cada vez mais importante desenvolver aplicações que não sejam apenas funcionais, mas que apresentem também características como organização, manutenibilidade, reutilização, escalabilidade e facilidade de evolução.

No sector dos transportes rodoviários, a utilização de sistemas informatizados pode contribuir significativamente para a modernização dos processos relacionados com a venda e gestão de bilhetes. No contexto moçambicano, o transporte interprovincial constitui um importante meio de mobilidade da população. Entretanto, processos predominantemente manuais podem provocar dificuldades na consulta de viagens, controlo de lugares, aquisição de bilhetes, gestão das informações dos passageiros e confirmação dos pagamentos.

Neste contexto, o presente projecto propõe o desenvolvimento de um **Sistema de Venda de Bilhetes de Autocarro Interprovincial**, concebido como uma aplicação web destinada a facilitar e automatizar o processo de aquisição de bilhetes. O sistema permitirá ao passageiro consultar viagens disponíveis, seleccionar a origem e o destino, visualizar informações sobre as viagens, fornecer os seus dados, efectuar o pagamento e receber um bilhete electrónico. O projecto anterior já estabelecia como funcionalidades centrais a consulta de viagens, a atribuição automática de assentos, o pagamento através do M-Pesa e a emissão de um bilhete electrónico com identificador único.

Além da implementação das funcionalidades do sistema, este projecto concentra-se na aplicação de conceitos fundamentais da Engenharia de Software, particularmente **arquitectura de software, princípios SOLID e Design Patterns**. A utilização destes conceitos permitirá estruturar o sistema de forma modular, facilitando a manutenção, reutilização e extensão das funcionalidades.

Deste modo, pretende-se demonstrar que a aplicação dos princípios SOLID e dos padrões de projecto pode contribuir para a construção de uma solução de software mais organizada e preparada para futuras alterações, como a introdução de novos métodos de pagamento, novos mecanismos de notificação ou novas funcionalidades de gestão.

1.2 Problematização

Os processos manuais de venda de bilhetes podem causar dificuldades na consulta de viagens, controlo de assentos, gestão dos passageiros e realização dos pagamentos. O projecto pretende responder a estas limitações através da digitalização do processo de compra de bilhetes.

Além disso, um sistema com várias funcionalidades pode tornar-se difícil de manter quando não possui uma arquitectura adequada. Por isso, torna-se necessário aplicar princípios e padrões que permitam organizar melhor os componentes do software.

1.3 Pergunta de pesquisa

De que forma a aplicação dos princípios SOLID e dos Design Patterns pode contribuir para o desenvolvimento de um Sistema de Venda de Bilhetes de Autocarro mais modular, reutilizável e fácil de manter?

1.4 Objectivos

1.4.1 Objectivo geral

Desenvolver e modelar um Sistema de Venda de Bilhetes de Autocarro Interprovincial, aplicando princípios SOLID e Design Patterns na definição da arquitectura e implementação do sistema, de modo a garantir maior modularidade, reutilização, manutenibilidade e facilidade de extensão.

1.4.2 Objectivos específicos

- ✓ Analisar os conceitos fundamentais da Engenharia de Software e da Arquitectura de Software aplicáveis ao sistema;
- ✓ Identificar e analisar os requisitos funcionais e não funcionais do Sistema;
- ✓ Modelar o sistema utilizando diagramas UML adequados;
- ✓ Aplicar os cinco princípios SOLID na organização dos componentes do sistema;
- ✓ Identificar os Design Patterns mais adequados às necessidades do sistema;
- ✓ Aplicar padrões criacionais, estruturais e comportamentais em diferentes componentes da solução;
- ✓ Definir uma arquitectura de software que permita separar adequadamente as responsabilidades do sistema;
- ✓ Implementar os principais módulos do sistema, incluindo consulta de viagens, compra de bilhetes, pagamentos, gestão de assentos e emissão de bilhetes;
- ✓ Integrar a base de dados e os serviços necessários ao funcionamento da aplicação;

- ✓ Avaliar, através de testes, o funcionamento do sistema e a aplicação dos princípios SOLID e dos Design Patterns;
- ✓ Analisar as vantagens e limitações da arquitectura proposta.

1.5 Justificativa

A escolha do **Sistema de Venda de Bilhetes de Autocarro Interprovincial** justifica-se pela sua relevância prática e pela possibilidade de aplicar, num único projecto, diferentes conceitos estudados na disciplina de Engenharia de Software.

O sistema apresenta um conjunto de funcionalidades suficientemente diversificado para permitir o estudo de arquitectura de software, modelação UML, princípios de design, SOLID e Design Patterns. Entre as funcionalidades identificadas no projecto encontram-se a gestão de autocarros, gestão de viagens, gestão de horários e preços, consulta de viagens, processamento de pagamentos, atribuição automática de assentos e emissão de bilhetes electrónicos.

Do ponto de vista académico, o projecto permite transformar conceitos teóricos de Engenharia de Software em soluções concretas. A aplicação do **SOLID** possibilita demonstrar como as responsabilidades podem ser distribuídas entre diferentes componentes, enquanto os **Design Patterns** permitem apresentar soluções reutilizáveis para problemas recorrentes no desenvolvimento de software.

A escolha deste sistema também se justifica pela necessidade de considerar aspectos como segurança, integridade dos dados, desempenho, suporte a múltiplos utilizadores e facilidade de manutenção. Estes aspectos já foram identificados entre os requisitos não funcionais do projecto, incluindo a protecção dos dados dos passageiros, a integridade das informações, o suporte a acessos simultâneos e a facilidade de manutenção.

Por fim, o projecto apresenta potencial para futuras extensões, como a integração com diferentes plataformas de pagamento, envio do bilhete por SMS, geração de QR Code e inclusão de novos serviços. Estas possibilidades tornam importante a utilização de uma arquitectura que permita adicionar funcionalidades sem exigir alterações excessivas nos componentes já existentes.

1.6 Metodologia

Será utilizada uma **metodologia de desenvolvimento incremental**, na qual o sistema será desenvolvido progressivamente por módulos. Esta abordagem permitirá analisar, implementar e testar as funcionalidades em diferentes etapas.

O desenvolvimento compreenderá as seguintes etapas:

a) Levantamento e análise dos requisitos: identificação das funcionalidades, regras de negócio, actores e requisitos não funcionais do sistema.

b) Fundamentação teórica: estudo dos conceitos de Engenharia de Software, Arquitectura de Software, princípios de design, SOLID e Design Patterns.

c) Modelagem do sistema: elaboração dos diagramas UML necessários para representar a estrutura e o comportamento da aplicação.

d) Definição da arquitectura: identificação dos componentes, responsabilidades, interfaces e relações entre as diferentes partes do sistema.

e) Aplicação dos princípios SOLID: análise da estrutura das classes e componentes, procurando reduzir responsabilidades excessivas, dependências desnecessárias e dificuldades de extensão.

f) Selecção e aplicação dos Design Patterns: escolha dos padrões criacionais, estruturais e comportamentais adequados aos problemas identificados no sistema.

g) Implementação: desenvolvimento dos principais componentes da aplicação, incluindo interface, lógica de negócio, acesso aos dados, pagamento, gestão de assentos e emissão de bilhetes.

h) Integração: integração dos diferentes módulos e da base de dados, bem como do serviço de pagamento, quando aplicável.

i) Testes e avaliação: realização de testes unitários e de integração, avaliação do funcionamento do sistema e análise da aplicação dos princípios SOLID e dos padrões seleccionados.

j) Documentação: organização dos resultados, análise das decisões arquitecturais, apresentação das limitações e elaboração da documentação final.

A abordagem incremental permite, portanto, desenvolver o sistema de forma progressiva e controlada. No projecto inicial foram definidos incrementos que começam pelo levantamento e modelagem, avançam para consulta e gestão, posteriormente para compra e pagamento, e terminam com testes, correcções e documentação.

CAPÍTULO II — FUNDAMENTAÇÃO TEÓRICA

2.1 Engenharia de Software

A Engenharia de Software compreende métodos e práticas utilizados para desenvolver sistemas de software de forma organizada, procurando garantir qualidade, facilidade de manutenção e capacidade de evolução. No presente projecto, estes conceitos serão aplicados nas fases de análise, modelação, implementação e testes do sistema.

2.2 Arquitectura de Software

A arquitectura de software define a organização dos principais componentes de um sistema e as relações existentes entre eles. Uma arquitectura adequada favorece a **modularidade, escalabilidade e manutenibilidade**, permitindo que componentes possam ser testados, substituídos ou modificados com menor impacto no restante sistema.

No Sistema de Venda de Bilhetes, a arquitectura será utilizada para organizar os componentes responsáveis pela interface, regras de negócio, acesso à base de dados e comunicação com serviços externos.

2.3 Princípios de Design

Os princípios de design são orientações que auxiliam na construção de sistemas mais organizados e flexíveis. Entre os aspectos importantes encontram-se a **abstracção, modularidade, baixo acoplamento e separação de responsabilidades**.

A aplicação destes princípios permite reduzir a complexidade do sistema e facilita futuras alterações. A modularidade, por exemplo, permite desenvolver ou substituir componentes individuais sem afectar necessariamente todo o sistema.

2.4 SOLID

SOLID é um conjunto de cinco princípios utilizados no desenvolvimento orientado a objectos para melhorar a organização e manutenção do código.

Os princípios são:

- ✓ **SRP — Single Responsibility Principle:** uma classe deve possuir uma responsabilidade principal;
- ✓ **OCP — Open/Closed Principle:** os componentes devem permitir extensão sem alterações desnecessárias;
- ✓ **LSP — Liskov Substitution Principle:** objectos derivados devem poder substituir os seus tipos base;

- ✓ **ISP — Interface Segregation Principle:** as interfaces devem ser específicas para as necessidades dos seus clientes;
- ✓ **DIP — Dependency Inversion Principle:** os componentes devem depender de abstracções em vez de implementações concretas.

Estes princípios serão aplicados posteriormente na modelação e implementação do sistema.

2.5 Design Patterns

Segundo **Gamma et al.**, os padrões de projecto representam soluções reutilizáveis para problemas recorrentes no desenvolvimento de software orientado a objectos. Os padrões não constituem programas completos, mas apresentam soluções que foram desenvolvidas e aperfeiçoadas através da experiência de desenvolvimento de sistemas.

Os autores destacam ainda que os padrões ajudam a explicar **por que determinada solução de projecto foi adoptada**, facilitando a transformação do modelo de análise em um projecto orientado a objectos reutilizável.

Além disso, os padrões fornecem um vocabulário comum para comunicar, documentar e discutir decisões de projecto entre os desenvolvedores.

2.6 Padrões Criacionais

Os padrões criacionais estão relacionados com os mecanismos utilizados para **criar objectos** de maneira flexível.

Neste projecto serão estudados:

- ✓ **Singleton**;
- ✓ **Factory Method**;
- ✓ **Builder**;
- ✓ **Abstract Factory**;
- ✓ **Prototype**.

A sua utilização poderá facilitar, por exemplo, a criação de diferentes tipos de objectos relacionados com pagamentos, bilhetes e outros componentes do sistema.

2.7 Padrões Estruturais

Os padrões estruturais tratam da **composição e dos relacionamentos entre objectos e classes**, permitindo organizar componentes de maior dimensão.

Neste projecto serão analisados:

- ✓ **Adapter**;

- ✓ **Facade**;
- ✓ **Decorator**;
- ✓ **Proxy**;
- ✓ Outros padrões considerados relevantes.

2.8 Padrões Comportamentais

Os padrões comportamentais estão relacionados com a **comunicação e distribuição de responsabilidades entre objectos**.

Serão estudados principalmente:

- ✓ **Strategy**;
- ✓ **Observer**;
- ✓ **State**;
- ✓ **Command**;
- ✓ Outros padrões aplicáveis.

A escolha de cada padrão será realizada de acordo com o problema que se pretende resolver, uma vez que os padrões devem ser seleccionados considerando a natureza específica do problema e não aplicados de forma indiscriminada.

CAPÍTULO III — ANÁLISE DO SISTEMA

3.1 Descrição

O sistema proposto é uma aplicação web para **venda de bilhetes de autocarro interprovincial**. O sistema permitirá aos passageiros consultar viagens disponíveis, seleccionar uma viagem, efectuar a compra e receber o respectivo bilhete electrónico.

Do lado administrativo, o sistema permitirá gerir informações relacionadas com autocarros, viagens, horários, preços, passageiros e bilhetes. Estas funcionalidades já fazem parte da definição inicial do projecto.

3.2 Problema

O processo tradicional de venda de bilhetes pode apresentar dificuldades na consulta de viagens, controlo dos lugares disponíveis, gestão dos passageiros e confirmação dos pagamentos.

Dessa forma, surge a necessidade de um sistema informatizado que centralize estas operações e permita realizar o processo de compra de forma mais organizada e eficiente.

Além da funcionalidade, o sistema deverá possuir uma estrutura que facilite a manutenção e a introdução de novas funcionalidades. Por isso, serão aplicados os princípios **SOLID** e, posteriormente, os **padrões criacionais** seleccionados.

3. 3 Requisitos funcionais

Os principais requisitos funcionais do sistema são:

- ✓ **RF01:** Permitir o registo e gestão de passageiros;
- ✓ **RF02:** Permitir o registo e gestão de autocarros;
- ✓ **RF03:** Permitir o registo e gestão de viagens;
- ✓ **RF04:** Permitir consultar viagens disponíveis;
- ✓ **RF05:** Permitir visualizar horários e preços;
- ✓ **RF06:** Permitir seleccionar uma viagem;
- ✓ **RF07:** Permitir efectuar a compra do bilhete;
- ✓ **RF08:** Permitir efectuar o pagamento;
- ✓ **RF09:** Permitir atribuir um assento ao passageiro;
- ✓ **RF10:** Emitir o bilhete electrónico;
- ✓ **RF11:** Permitir ao administrador gerir as informações do sistema.

O projecto inicial também define funcionalidades relacionadas com consulta de viagens, pagamento, atribuição automática de assentos e emissão do bilhete electrónico.

3.4 Requisitos não funcionais

O sistema deverá atender aos seguintes requisitos não funcionais:

- **Segurança:** proteger os dados dos utilizadores;
- **Desempenho:** responder às solicitações num período adequado;
- **Disponibilidade:** permitir acesso ao sistema sempre que necessário;
- **Integridade:** garantir a consistência dos dados;
- **Usabilidade:** apresentar uma interface simples e compreensível;
- **Manutenibilidade:** facilitar alterações e correcções;
- **Escalabilidade:** permitir o crescimento do sistema.

Estes aspectos estão alinhados com os requisitos não funcionais definidos na proposta inicial do projecto.

3.5 Actores

Os principais actores identificados são:

Passageiro

É o utilizador que consulta viagens, fornece os seus dados, selecciona uma viagem, efectua o pagamento e recebe o bilhete electrónico.

Administrador

É responsável pela gestão do sistema, incluindo autocarros, viagens, horários, preços, passageiros e bilhetes.

Serviço de Pagamento

Representa o serviço externo responsável pelo processamento e confirmação do pagamento.

3.6 Casos de uso

Os principais casos de uso são:

Actor	Casos de uso
Passageiro	Consultar viagens
	Seleccionar viagem
	Informar dados
	Efectuar pagamento
	Receber bilhete
Administrador	Gerir passageiros
	Gerir autocarros
	Gerir viagens
	Gerir horários
	Gerir preços
	Consultar bilhetes
Serviço de Pagamento	Processar pagamento
	Confirmar pagamento

O fluxo principal da compra será:

Consultar viagem → Seleccionar viagem → Informar dados → Efectuar pagamento → Confirmar pagamento → Atribuir assento → Emitir bilhete.

Este fluxo mantém a lógica definida anteriormente no projecto, segundo a qual a confirmação do pagamento antecede a atribuição do assento e a emissão do bilhete.

CAPÍTULO IV — MODELAGEM DO SISTEMA

A modelagem do sistema será realizada utilizando a **UML (Unified Modeling Language)**, permitindo representar a estrutura, o comportamento e a implantação do Sistema de Venda de Bilhetes de Autocarro Interprovincial.

Como nesta fase o projecto está centrado nos **padrões criacionais**, a modelagem dará especial atenção às classes e às relações que posteriormente permitirão aplicar **Singleton, Factory Method, Builder, Abstract Factory e Prototype**.

4.1 Diagramas Estruturais

4.1.1 Diagrama de Classes

O Diagrama de Classes representa as principais classes do sistema, os seus atributos, métodos e relacionamentos.

As principais classes identificadas são:

- **Passageiro** — representa o cliente que compra o bilhete;
- **Autocarro** — representa o veículo utilizado na viagem;
- **Viagem** — contém as informações da viagem;
- **Bilhete** — representa o bilhete adquirido pelo passageiro;
- **Pagamento** — representa o processo de pagamento;
- **Assento** — representa o lugar atribuído ao passageiro;
- **Administrador** — responsável pela gestão do sistema.

Uma representação inicial pode ser organizada da seguinte forma:

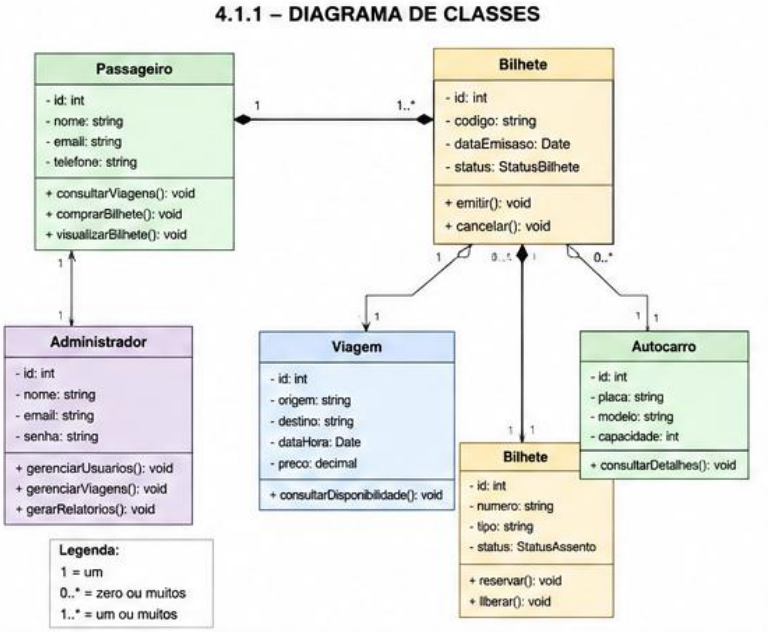


Figura 1:Diagrama de Classes

Posteriormente, serão acrescentadas as classes necessárias à aplicação dos padrões criacionais.

4.1.2 Diagrama de Componentes

O Diagrama de Componentes representa a organização dos principais módulos do sistema e as suas dependências.

A arquitectura inicial poderá ser dividida em:

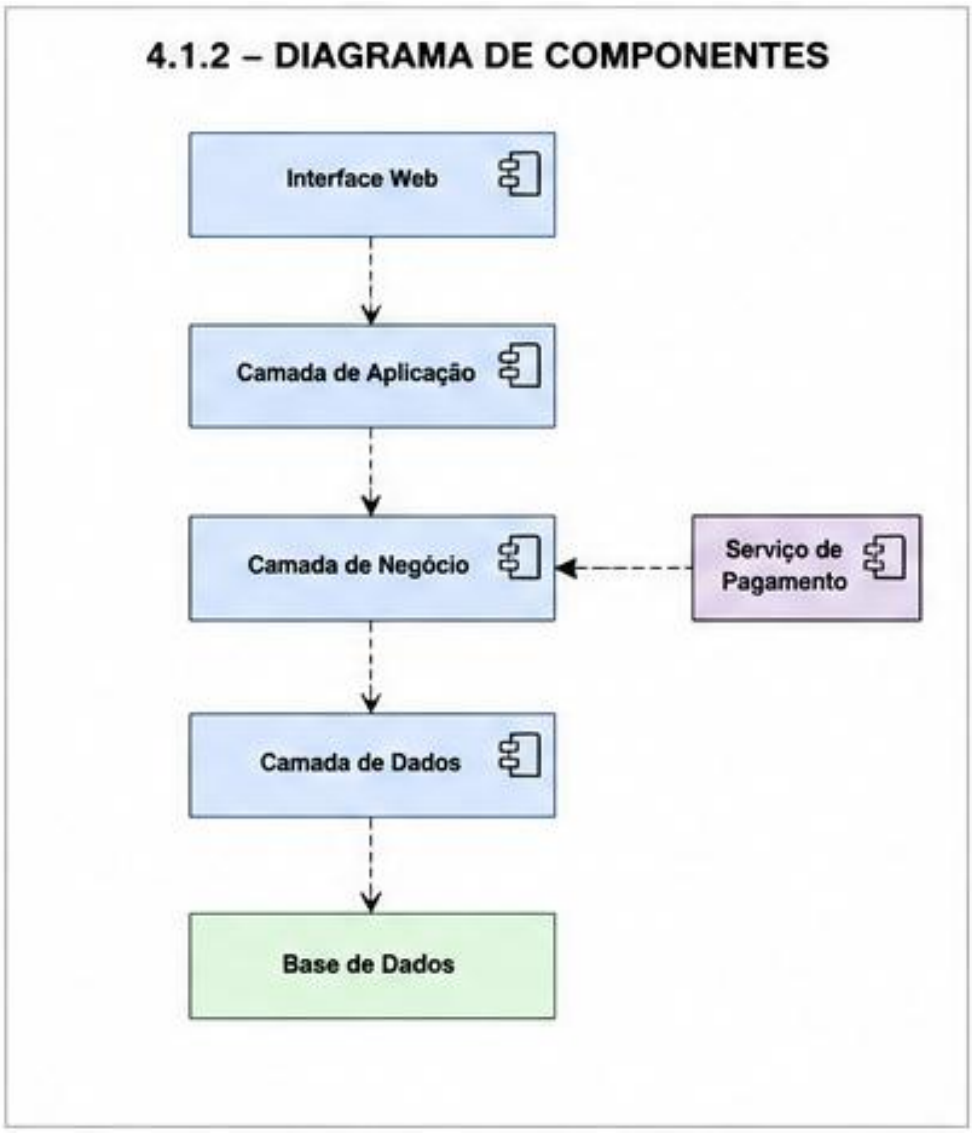


Figura 2: Diagrama de Componentes

Esta organização permite separar as responsabilidades e reduzir o acoplamento entre os componentes.

4.2 Diagramas Comportamentais

4.2.1 Diagrama de Sequência

O Diagrama de Sequência representa a ordem das interacções entre os componentes durante a realização de uma operação.

Para o processo de compra, a sequência principal será:

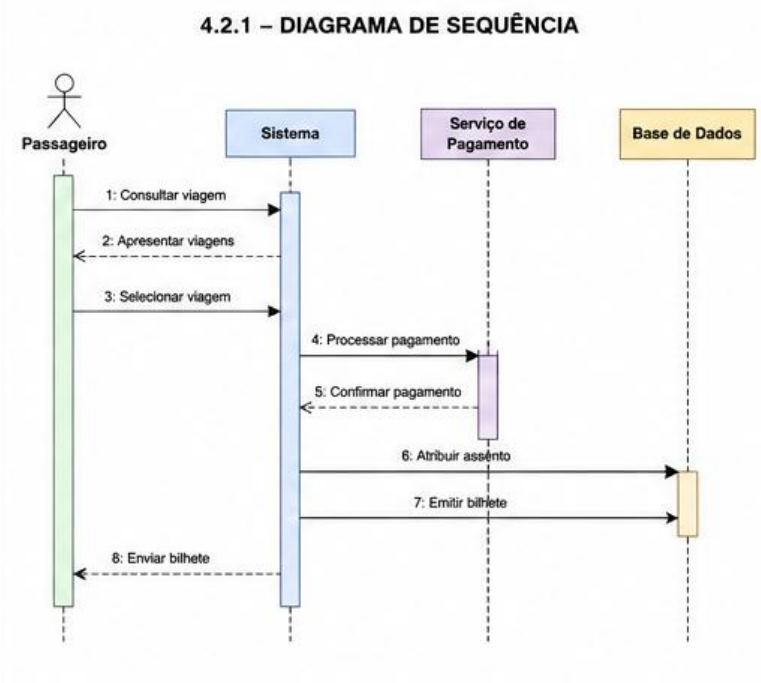


Figura 3: Diagrama de Sequência

4.2.2 Diagrama de Actividades

O processo principal poderá ser representado pelas seguintes actividades:

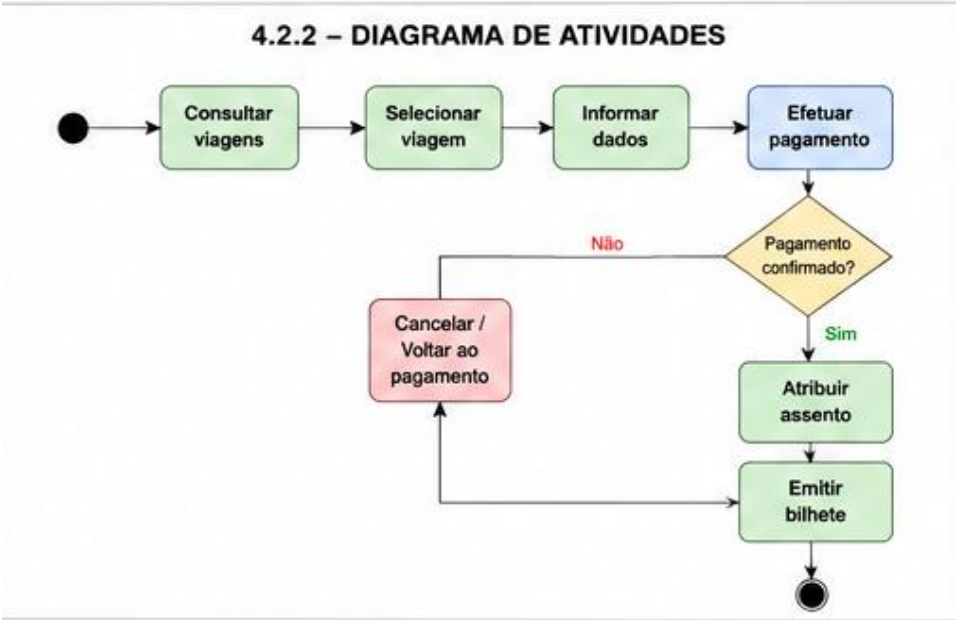


Figura 4: Diagrama de Actividades

Início → Consultar viagens → Seleccionar viagem → Informar dados → Efectuar pagamento → Pagamento confirmado?

- **Não:** cancelar/voltar ao pagamento;
- **Sim:** atribuir assento → emitir bilhete → finalizar.

4.2.3 Diagrama de Estados

O bilhete poderá possuir diferentes estados:

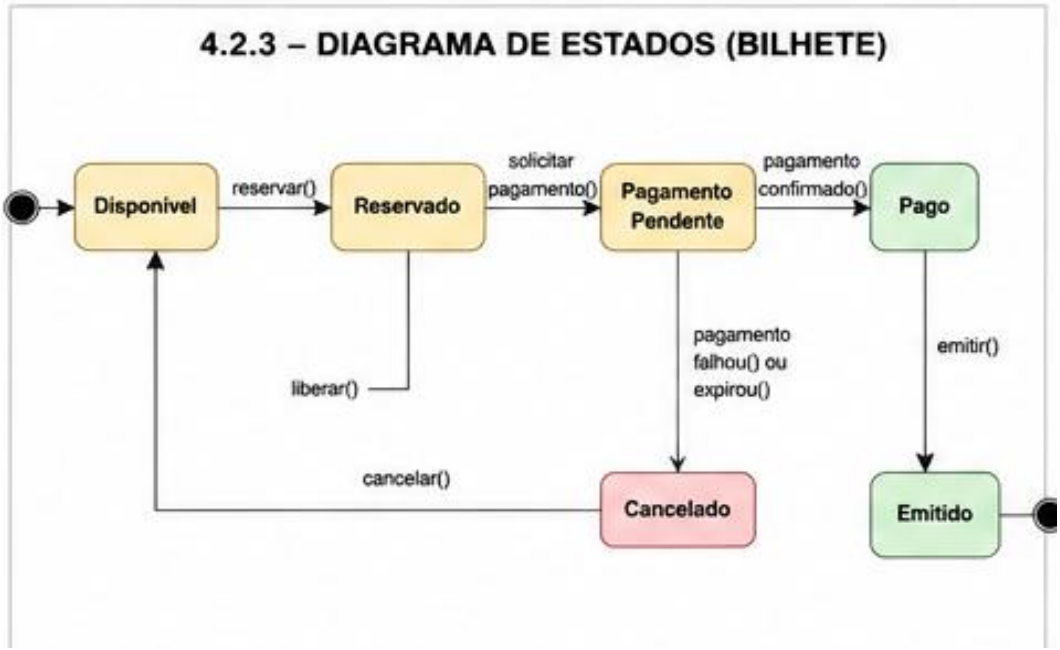


Figura 5: Diagrama de Estados

Caso o pagamento não seja confirmado, a operação poderá retornar ao estado anterior ou ser cancelada.

4.2.4 Diagrama de Comunicação

O Diagrama de Comunicação representa a troca de mensagens entre os objectos durante a compra.

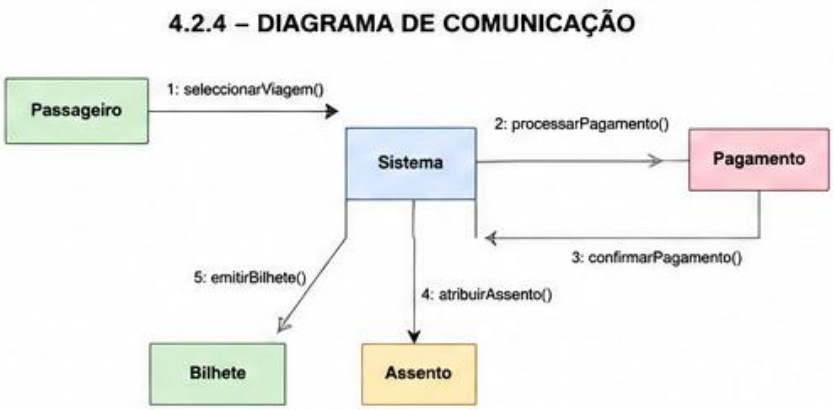


Figura 6: Diagrama de Comunicação

4.2.5 Diagrama de Interacção

O diagrama de interacção representa a colaboração entre os objectos envolvidos numa determinada operação.

No processo de compra, participam principalmente:

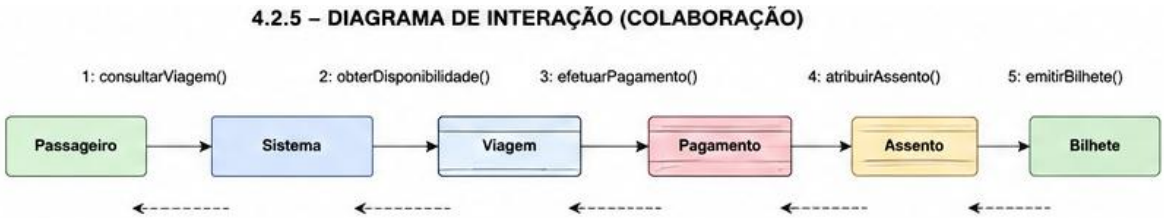


Figura 7: Diagrama de Interacção

Passageiro → Sistema → Viagem → Pagamento → Assento → Bilhete.

Esta interacção será importante para identificar posteriormente onde os padrões criacionais podem ser aplicados.

4.3 Diagrama de Implantação

O Diagrama de Implantação representa a distribuição física dos principais componentes do sistema.

Uma estrutura inicial será:

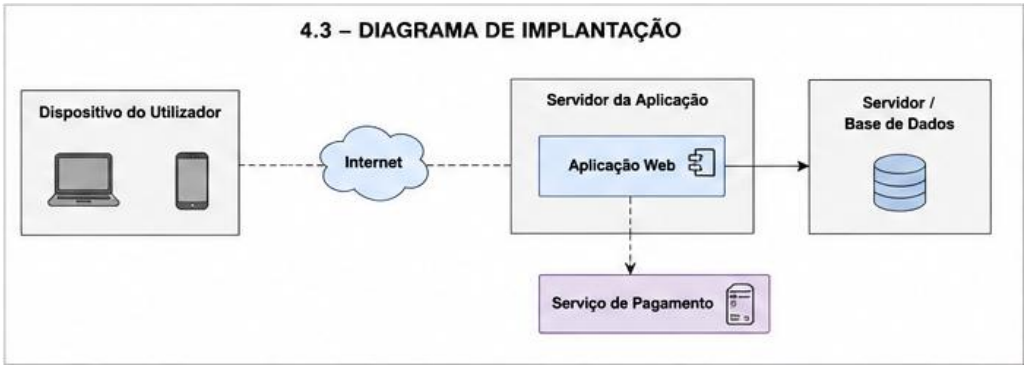


Figura 8: Diagrama de Implantação

O utilizador poderá aceder ao sistema através de um computador ou dispositivo móvel, enquanto a aplicação e a base de dados serão executadas no ambiente de servidor.

CAPÍTULO V — APLICAÇÃO DOS PRINCÍPIOS SOLID

Neste capítulo são aplicados os cinco princípios **SOLID** ao Sistema de Venda de Bilhetes de Autocarro Interprovincial. O objectivo é melhorar a organização das classes, reduzir o acoplamento e facilitar a manutenção e evolução do sistema.

5.1 SRP — Single Responsibility Principle

O **Single Responsibility Principle (Princípio da Responsabilidade Única)** estabelece que uma classe deve possuir uma responsabilidade principal. Dessa forma, uma alteração numa determinada funcionalidade deve afectar apenas as classes relacionadas com essa responsabilidade.

No sistema proposto, por exemplo, uma única classe não deverá ser responsável simultaneamente por **processar pagamentos, guardar dados na base de dados e emitir bilhetes**.

Exemplo de estrutura inadequada



Neste caso, a classe concentra várias responsabilidades, tornando o sistema mais difícil de testar e manter.

Aplicação do SRP

As responsabilidades serão separadas em diferentes classes:



Assim, cada componente concentra-se numa responsabilidade específica.

Aplicação no projecto

No Sistema de Venda de Bilhetes, o SRP será aplicado principalmente na separação das responsabilidades relacionadas com:

- Gestão de passageiros;
- Gestão de viagens;
- Processamento de pagamentos;
- Gestão de assentos;
- Emissão de bilhetes;
- Acesso à base de dados.

Esta separação também facilitará a aplicação posterior dos **padrões criacionais**, uma vez que as classes responsáveis pela criação de objectos poderão ser isoladas das restantes regras de negócio.

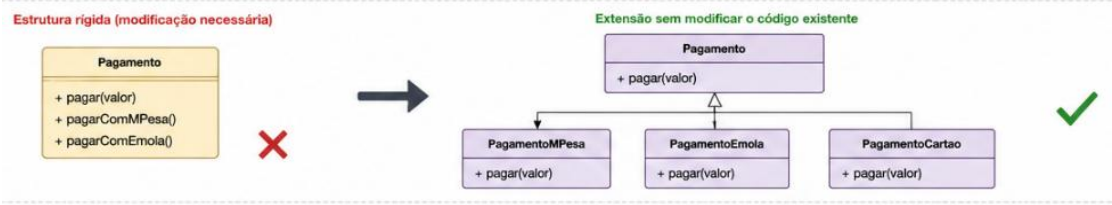
Resultado esperado: menor acoplamento, maior organização do código e maior facilidade de manutenção.

5.2 OCP — Open/Closed Principle

O **Open/Closed Principle** estabelece que os componentes de software devem estar **abertos para extensão, mas fechados para modificação**.

No projecto, este princípio será particularmente importante na implementação dos métodos de pagamento. O sistema deverá permitir adicionar novos métodos de pagamento sem modificar excessivamente o código existente.

Por exemplo:

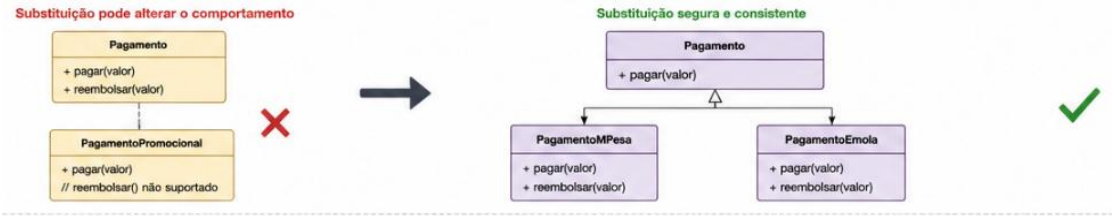


Dessa forma, um novo método de pagamento poderá ser acrescentado através de uma nova implementação, sem alterar as funcionalidades já existentes.

5.3 LSP — Liskov Substitution Principle

O **Liskov Substitution Principle** determina que uma classe derivada deve poder substituir a sua classe base sem alterar o comportamento esperado do sistema.

No projecto, diferentes tipos de pagamento deverão respeitar o mesmo contrato definido pela abstracção de pagamento.



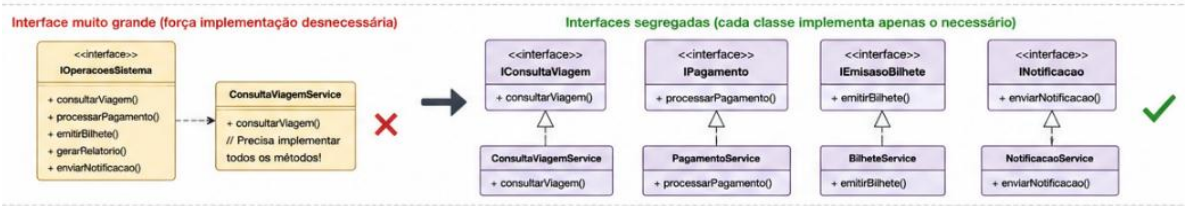
Assim, o sistema poderá trabalhar com diferentes implementações sem depender directamente de uma implementação específica.

5.4 ISP — Interface Segregation Principle

O **Interface Segregation Principle** estabelece que uma classe não deve ser obrigada a implementar métodos que não necessita.

Em vez de criar uma interface demasiado grande, as funcionalidades poderão ser divididas em interfaces menores e específicas.

Por exemplo:

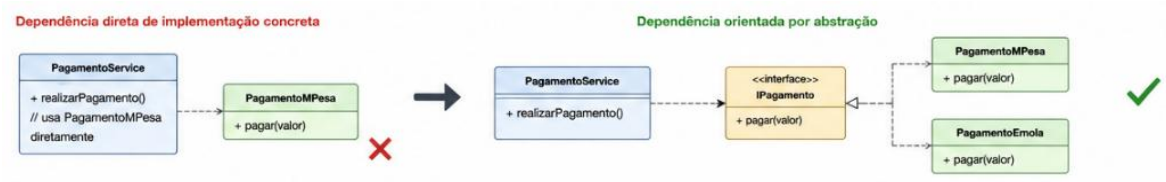


Dessa forma, cada classe implementará apenas as operações necessárias para a sua responsabilidade.

5.5 DIP — Dependency Inversion Principle

O **Dependency Inversion Principle** estabelece que os módulos de alto nível não devem depender directamente de módulos de baixo nível. Ambos devem depender de abstracções.

No sistema, por exemplo, o serviço de pagamento não deverá depender directamente da implementação do M-Pesa.



Assim, futuramente será possível substituir ou acrescentar métodos de pagamento sem alterar significativamente a lógica principal do sistema.

6.1 Singleton

O **Singleton** é um padrão criacional utilizado quando se pretende garantir que uma determinada classe possua **uma única instância**, disponibilizando um ponto de acesso a essa instância.

No contexto do Sistema de Venda de Bilhetes de Autocarro, o Singleton poderá ser considerado para componentes que devem possuir uma única instância durante a execução da aplicação, como um gestor central de configuração ou de conexão.

Estrutura proposta



Figura 9: Singleton

A classe DatabaseManager mantém a sua própria instância e disponibiliza o método getInstance() para que os restantes componentes possam aceder à mesma instância.

Aplicação no sistema

Neste projecto, o Singleton **será analisado**, mas a sua utilização definitiva dependerá da arquitectura e da tecnologia escolhida para a implementação. Não será considerado automaticamente obrigatório apenas por ser um padrão criacional.

Vantagens

- Controla a quantidade de instâncias de uma classe;
- Disponibiliza um ponto de acesso global;

- Pode evitar a criação desnecessária de vários objectos.

Limitações

- Pode introduzir dependência global entre componentes;
- Pode dificultar determinados testes;
- Deve ser utilizado apenas quando existir uma necessidade real de uma única instância.

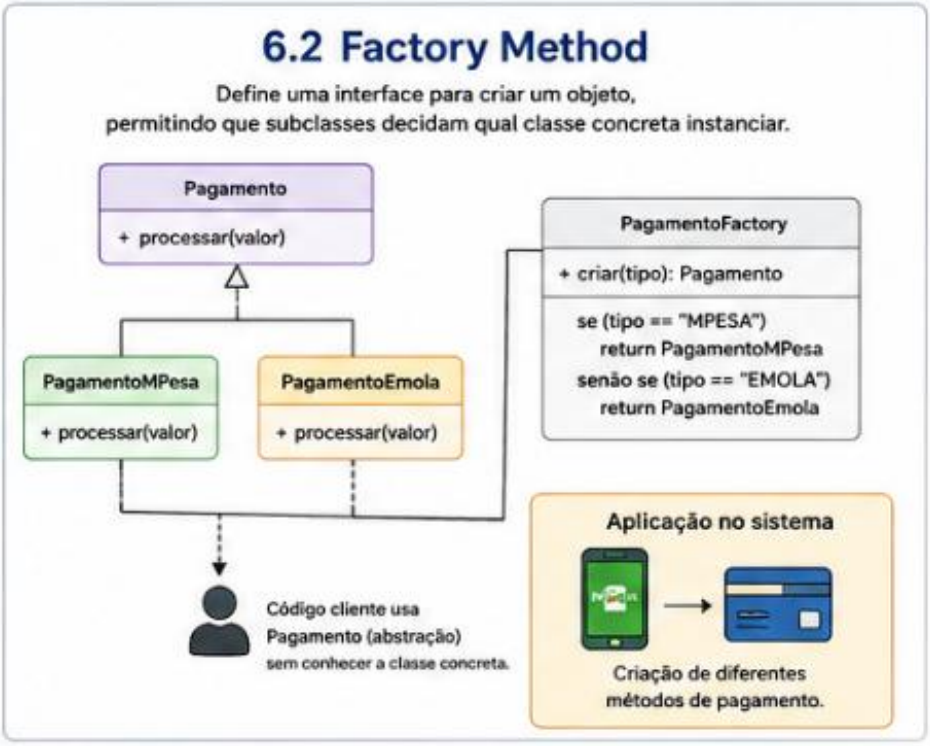
Decisão: A analisar para possível utilização, dependendo da arquitectura final.

6.2 Factory Method

O **Factory Method** é um padrão criacional que permite delegar às subclasses a criação dos objectos que serão utilizados pelo sistema.

No nosso projecto, este padrão apresenta uma aplicação interessante na criação de diferentes tipos de pagamento.

Por exemplo:



A lógica principal do sistema poderá trabalhar com a abstracção Pagamento, sem depender directamente de PagamentoMPesa ou PagamentoEmola.

Aplicação no sistema

Este padrão é um dos **candidatos mais fortes para implementação**, porque o sistema poderá futuramente suportar diferentes métodos de pagamento.

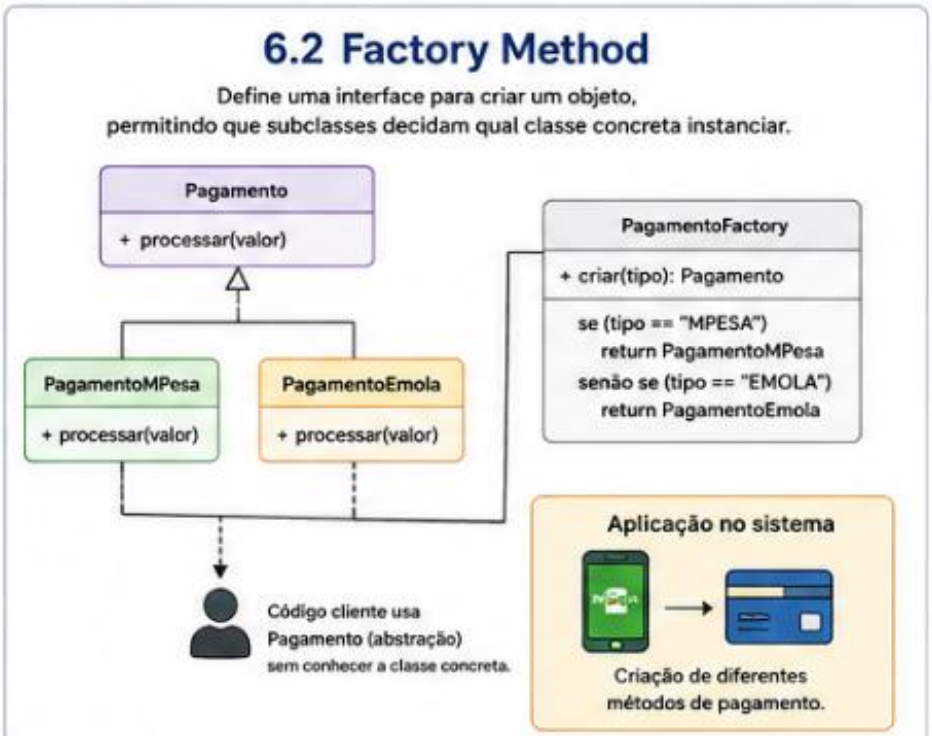
Decisão: Provavelmente será seleccionado.

6.3 Builder

O **Builder** é utilizado para construir objectos complexos de forma gradual, permitindo separar o processo de construção do objecto final.

No nosso sistema, o padrão apresenta uma aplicação natural na criação do **bilhete electrónico**, pois o bilhete pode reunir diversas informações:

- Passageiro;
- Viagem;
- Origem;
- Destino;
- Data;
- Horário;
- Assento;
- Preço;
- Pagamento;
- Código do bilhete.



Esta abordagem torna a criação do bilhete mais legível e evita construtores com muitos parâmetros.

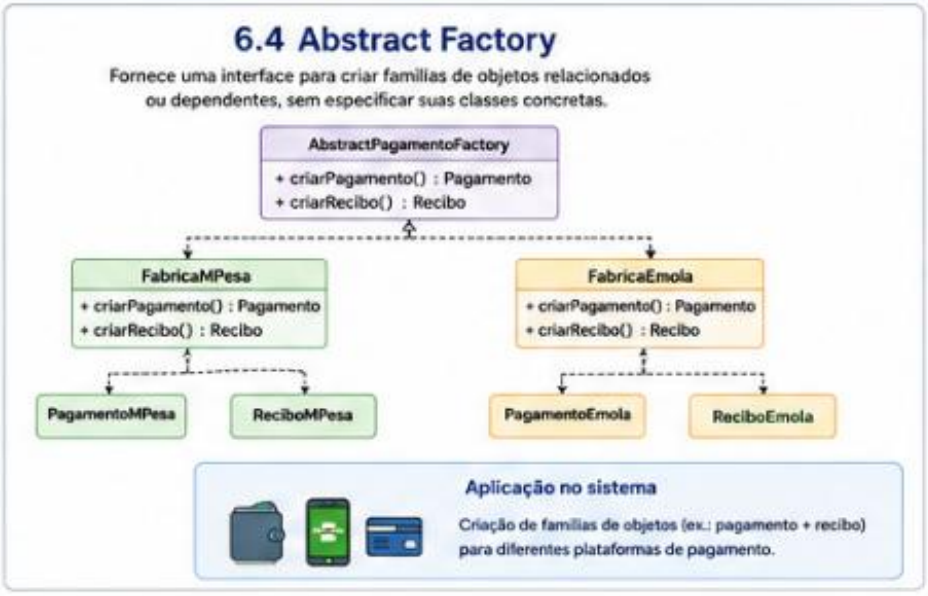
Decisão: Será provavelmente seleccionado.

6.4 Abstract Factory

O **Abstract Factory** fornece uma interface para criação de **famílias de objectos relacionados**, sem que o código cliente precise conhecer as suas implementações concretas.

No nosso sistema, este padrão poderia ser utilizado caso existissem diferentes famílias de serviços relacionados com uma determinada plataforma de pagamento.

Por exemplo:



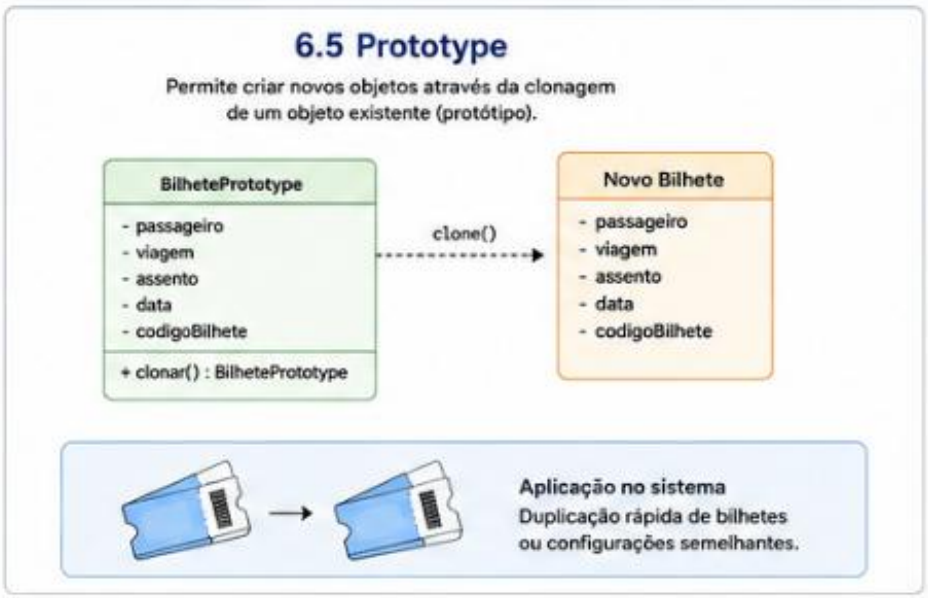
Entretanto, é importante não introduzir este padrão apenas para aumentar o número de padrões utilizados. Se o sistema não possuir famílias de objectos relacionadas que justifiquem a sua utilização, o padrão poderá ser apenas estudado e posteriormente rejeitado.

Decisão: A analisar; poderá não ser implementado.

6.5 Prototype

O **Prototype** permite criar novos objectos através da clonagem de um objecto existente.

No sistema de venda de bilhetes, poderia ser utilizado quando existisse a necessidade de criar rapidamente objectos semelhantes a partir de um modelo existente.



Contudo, para o sistema actualmente definido, não existe ainda uma necessidade clara que justifique a clonagem de bilhetes ou outros objectos.

Por esse motivo, o Prototype será estudado, mas **não será automaticamente incluído na implementação**.

Decisão: Provavelmente não seleccionado, salvo se surgir uma necessidade durante a implementação.

6.6 Análise comparativa

Padrão	Aplicação prevista	Decisão
Singleton	Gestão de instância única	Analisar
Factory Method	Criação de métodos de pagamento	**Seleccionar**
Builder	Construção do bilhete electrónico	**Seleccionar**
Abstract Factory	Famílias de serviços relacionados	Analisar
Prototype	Clonagem de objectos	Não prioritário

6.7 Selecção dos padrões

Após a análise inicial, os padrões que apresentam maior correspondência com os requisitos do sistema são:

Factory Method

Será utilizado para facilitar a criação de diferentes tipos de pagamento.

Builder

Será utilizado para construir o bilhete electrónico de forma organizada.

Singleton

Poderá ser utilizado para um componente que realmente necessite de uma única instância, dependendo da arquitectura final.

O **Abstract Factory** e o **Prototype** permanecerão na fundamentação teórica, mas a sua implementação dependerá da existência de uma necessidade concreta.

6.8 Justificação da selecção

A selecção dos padrões será baseada nos **problemas reais identificados no sistema**, e não na quantidade de padrões utilizados.

Dessa forma, o projecto demonstra que os padrões criacionais são ferramentas para solucionar problemas de projecto e não simplesmente componentes que devem ser adicionados ao código.

CAPÍTULO IX — ARQUITECTURA FINAL

Neste capítulo é apresentada a arquitectura final proposta para o **Sistema de Venda de Bilhetes de Autocarro Interprovincial**, considerando os requisitos identificados, os princípios SOLID e os padrões criacionais seleccionados.

9.1 Arquitectura proposta

A arquitectura proposta adopta uma organização em camadas, permitindo separar as responsabilidades do sistema e reduzir o acoplamento entre os seus componentes.

A estrutura será composta por:

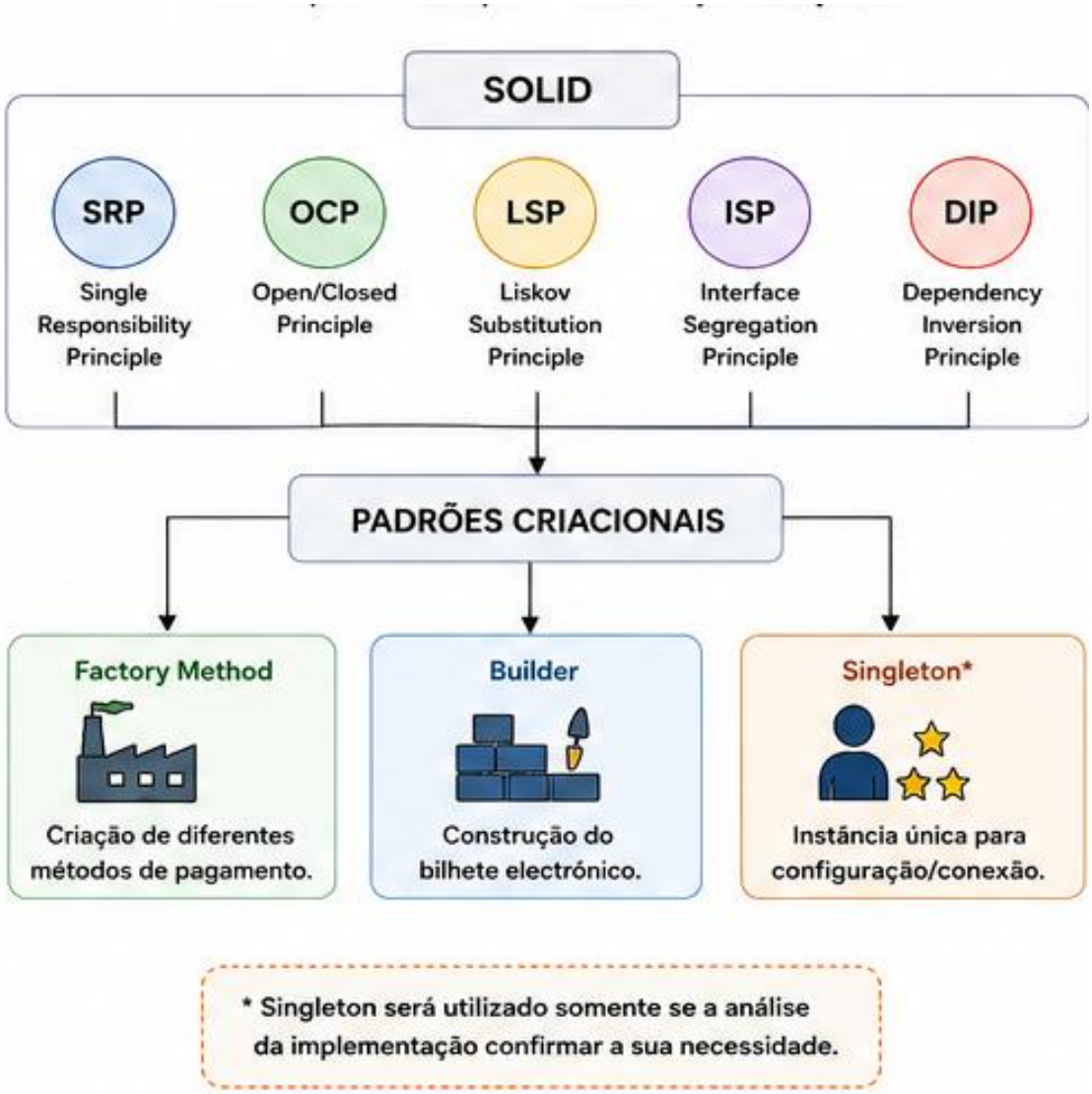


A separação em componentes permite maior modularidade e facilita a manutenção e evolução do sistema, aspecto relacionado aos princípios de arquitectura abordados no manual.

9.2 Relação SOLID + Patterns

Os princípios SOLID serão utilizados para organizar as responsabilidades, enquanto os padrões criacionais serão utilizados para resolver problemas específicos relacionados com a criação de objectos.

A relação principal será:



O **Factory Method** permitirá criar diferentes tipos de pagamento, enquanto o **Builder** será utilizado na construção do bilhete electrónico.

O **Singleton** será utilizado somente se a análise da implementação confirmar a necessidade de uma única instância.

9.3 Componentes

Os principais componentes da arquitectura serão:

Interface Web

Responsável pela interacção com os utilizadores, permitindo consultar viagens, efectuar compras e visualizar bilhetes.

Controllers

Recebem as solicitações provenientes da interface e encaminham-nas para os serviços correspondentes.

Services

Contêm a lógica das operações do sistema, como compra, pagamento, gestão de viagens e emissão de bilhetes.

Entidades

Representam os principais elementos do domínio:

- Passageiro;
- Viagem;
- Autocarro;
- Bilhete;
- Assento;
- Pagamento.

Repositories

Responsáveis pelo acesso e persistência dos dados na base de dados.

Factories e Builders

Serão responsáveis pela criação de determinados objectos, de acordo com os padrões seleccionados.

Serviços externos

Representam recursos externos, principalmente os serviços relacionados com pagamentos.

9.4 Fluxo de comunicação

O processo principal de compra de um bilhete seguirá aproximadamente o seguinte fluxo:



Neste fluxo, o **Factory Method** participa na criação do serviço de pagamento e o **Builder** participa na construção do bilhete.

9.5 Decisões arquitecturais

As principais decisões arquitecturais são:

1. **Separar as responsabilidades** entre diferentes camadas;
2. **Reduzir o acoplamento** entre os componentes;
3. Utilizar **abstracções e interfaces**, de acordo com os princípios SOLID;
4. Utilizar **Factory Method** para criação de diferentes métodos de pagamento;
5. Utilizar **Builder** para construção do bilhete electrónico;
6. Avaliar o **Singleton** antes da sua implementação;
7. Não utilizar padrões estruturais ou comportamentais apenas por obrigação;
8. Manter a arquitectura preparada para futuras extensões.

Esta abordagem está alinhada com a ideia de utilizar padrões como soluções reutilizáveis para problemas de projecto e como forma de comunicar decisões de arquitectura e design.

CAPÍTULO X — IMPLEMENTAÇÃO

10.1 Tecnologias

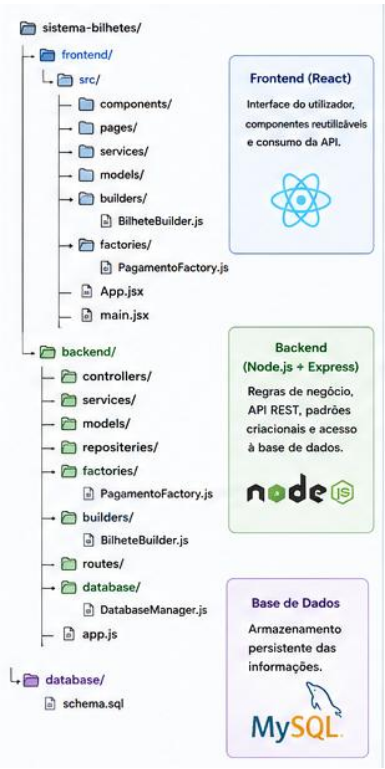
Para a implementação do Sistema de Venda de Bilhetes de Autocarro Interprovincial serão utilizadas tecnologias web modernas:

- **JavaScript** — linguagem principal;
- **React.js** — desenvolvimento da interface;
- **Node.js** — ambiente de execução do backend;
- **Express.js** — desenvolvimento da API;
- **MySQL** — sistema de gestão da base de dados;
- **HTML5 e CSS3** — estrutura e apresentação;
- **UML** — modelação do sistema;
- **Git/GitHub** — controlo de versões.

A arquitectura será dividida em **frontend** e **backend**, permitindo separar a interface da lógica de negócio e do acesso à base de dados.

10.2 Estrutura do projecto

A estrutura poderá ser organizada da seguinte forma:



Esta organização permite aplicar o **SRP**, separando componentes de acordo com as suas responsabilidades.

10.3 Implementação

10.3.1 React

O React será responsável pela interface do sistema.

Por exemplo, a página de consulta de viagens poderá ser composta por componentes:



O React permitirá dividir a interface em componentes reutilizáveis.

10.3.2 Factory Method

O padrão **Factory Method** será utilizado para criar diferentes tipos de pagamento.

Interface base

Em JavaScript, podemos utilizar uma classe base:

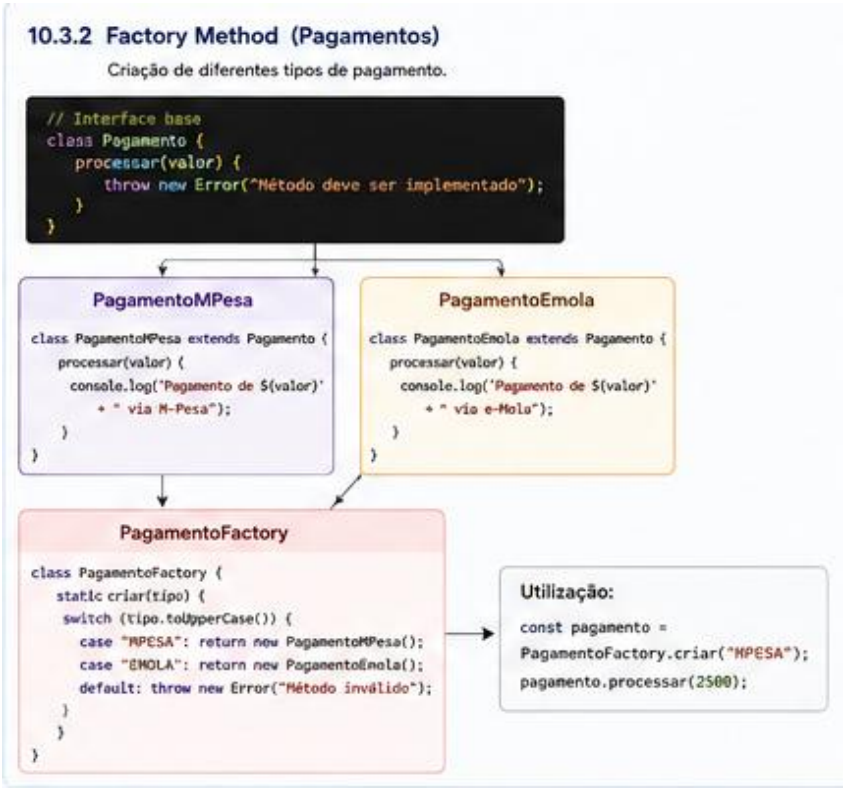


Figura 10: Factory Method

Assim, o restante sistema não precisa conhecer directamente as classes concretas.

10.4 Builder

O padrão **Builder** será utilizado para construir o bilhete electrónico.



10.5 Integração React + Backend

A arquitectura completa ficará:

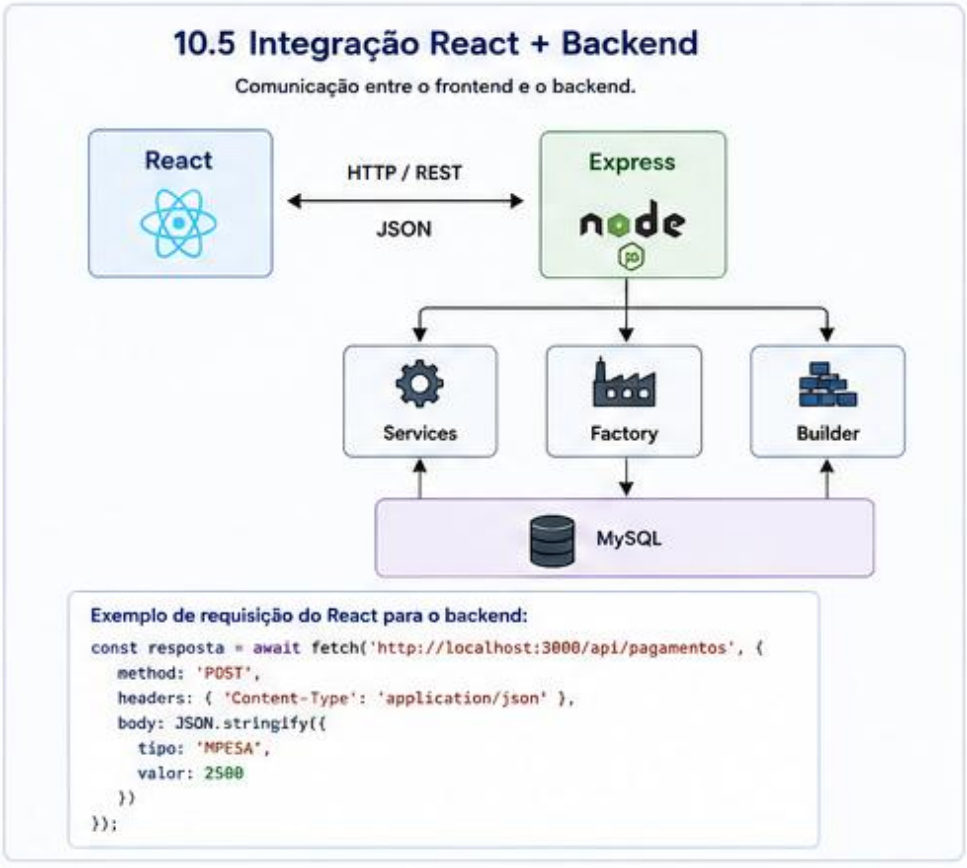


Figura 11: Integração React + Backend

O backend recebe a solicitação e utiliza o PagamentoFactory para criar o método de pagamento correspondente.

10.6 Base de dados

A aplicação utilizará **MySQL** para armazenar os dados.

As principais tabelas serão:

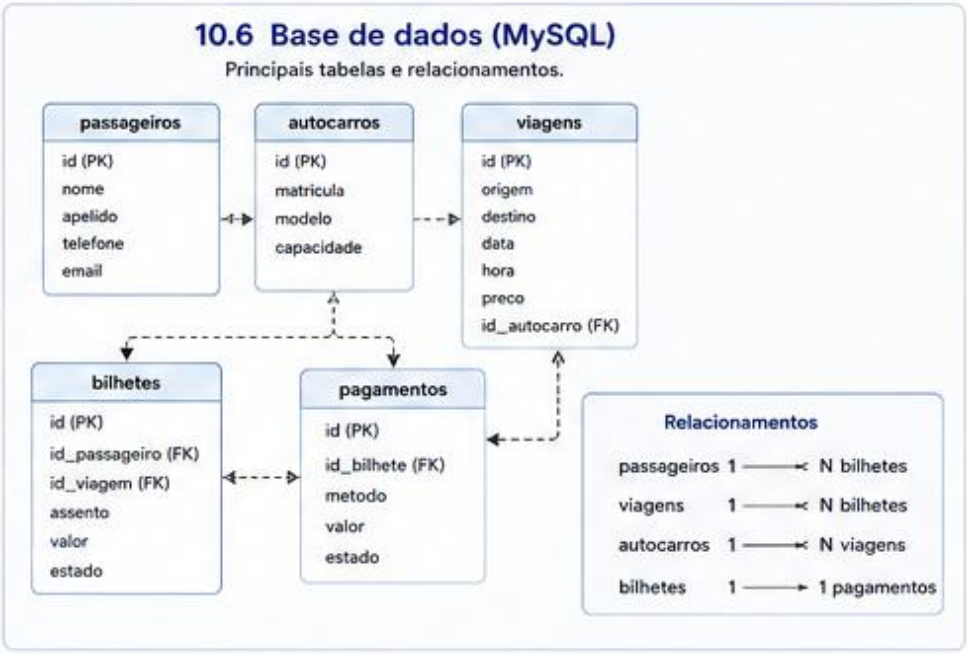


Figura 12: Base de dados

10.7 Singleton

O **Singleton** será analisado principalmente no backend para verificar se existe necessidade de uma única instância do gestor de conexão com a base de dados.

Por exemplo:



Figura 13: Singleton

Entretanto, **não vamos assumir que o Singleton é obrigatório**. Se a biblioteca de conexão escolhida já fornecer um mecanismo adequado de pool/conexões, poderemos concluir que não é necessário implementar o Singleton manualmente.

Arquitectura tecnológica definitiva



Com esta alteração, o projecto passa a ter uma stack coerente:

React + JavaScript → Node.js/Express → MySQL, e os padrões **Factory Method + Builder** serão implementados no backend, onde fazem mais sentido arquitecturalmente.

CAPÍTULO XI — TESTES E AVALIAÇÃO

Este capítulo apresenta os procedimentos utilizados para verificar o funcionamento do **Sistema de Venda de Bilhetes de Autocarro Interprovincial** e avaliar a aplicação dos princípios **SOLID** e dos padrões criacionais seleccionados.

A avaliação será realizada de forma progressiva, começando pelos componentes individuais e terminando com a integração do sistema.

11.1 Testes unitários

Os testes unitários serão utilizados para verificar individualmente as principais classes e componentes do sistema.

Factory Method

Será verificado se a PagamentoFactory cria correctamente cada tipo de pagamento.

Teste	Entrada	Resultado esperado
Criar M-Pesa	"MPESA"	Instância de PagamentoMPesa
Criar e-Mola	"EMOLA"	Instância de PagamentoEmola
Método inválido	"XYZ"	Erro de método inválido

Exemplo:

```
const pagamento = PagamentoFactory.criar("MPESA");
console.assert(
   pagamento instanceof PagamentoMPesa
);
```

Builder

Será verificado se o BilheteBuilder consegue construir correctamente um bilhete.

Teste	Resultado esperado
Adicionar passageiro	Passageiro armazenado
Adicionar viagem	Viagem armazenada
Adicionar assento	Assento armazenado
Adicionar pagamento	Método armazenado
Executar build()	Bilhete criado

11.2 Testes de integração

Os testes de integração verificarão a comunicação entre os diferentes componentes.

O principal fluxo a testar será:

11.3 Avaliação SOLID

A aplicação dos princípios SOLID será avaliada através da análise da estrutura do código.

Princípio	Critério de avaliação
SRP	Cada classe possui uma responsabilidade principal
OCP	Novos métodos de pagamento podem ser adicionados sem modificar excessivamente o código existente
LSP	Implementações de pagamento podem substituir a abstracção de pagamento
ISP	Interfaces não obrigam componentes a implementar funcionalidades desnecessárias
DIP	Componentes de alto nível dependem de abstracções

Exemplo

O PagamentoService não deverá criar directamente:

new PagamentoMPesa();

Em vez disso, deverá utilizar:

PagamentoFactory.criar("MPESA");

Isso reduz a dependência directa de uma implementação concreta.

11.4 Avaliação dos padrões

Os padrões serão avaliados considerando se realmente solucionam os problemas para os quais foram seleccionados.

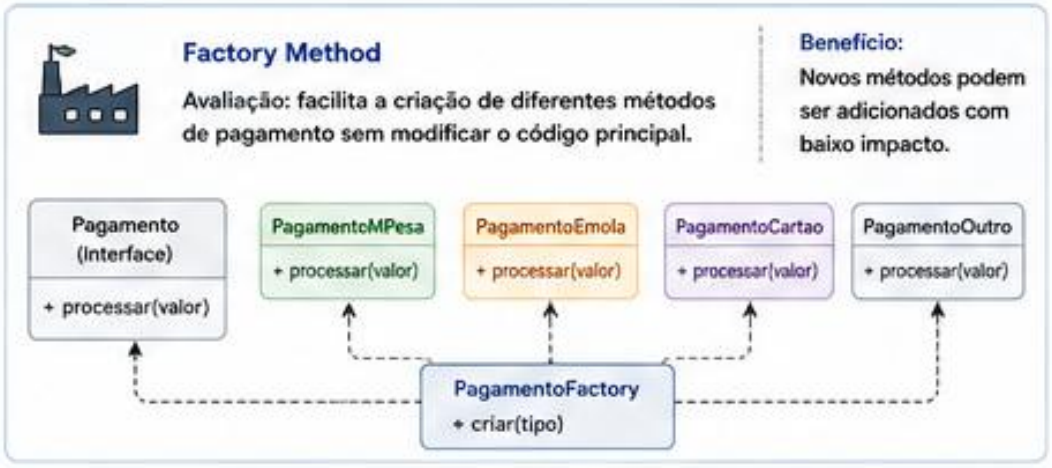


Figura 14: Avaliação dos padrões

11.5 Resultados

Os resultados dos testes serão registados numa tabela semelhante à seguinte:

Nº	Funcionalidade	Resultado esperado	Resultado obtido	Estado
1	Criar pagamento M-Pesa	Pagamento criado	Pagamento criado	Aprovado
2	Criar pagamento e-Mola	Pagamento criado	Pagamento criado	Aprovado
3	Método inválido	Apresentar erro	Apresentar erro	Aprovado
4	Construir bilhete	Bilhete criado	Bilhete criado	Aprovado
5	Guardar bilhete	Dados persistidos	Dados persistidos	Aprovado
6	Consultar viagem	Dados apresentados	Dados apresentados	Aprovado
7	Comprar bilhete	Compra concluída	Compra concluída	Aprovado

Nota: os resultados finais seram preenchidos depois de executarmos efectivamente os testes. Não devemos apresentar resultados como "Aprovado" se ainda não foram realizados.

CAPÍTULO XII — CONCLUSÃO

O presente projecto permitiu analisar e aplicar conceitos fundamentais de **Engenharia de Software, Arquitectura de Software, princípios SOLID e Design Patterns** no desenvolvimento de um Sistema de Venda de Bilhetes de Autocarro Interprovincial.

A análise realizada demonstrou a importância de organizar o sistema de forma modular, separando as responsabilidades entre os diferentes componentes. A aplicação dos princípios SOLID contribuiu para reduzir o acoplamento e melhorar a possibilidade de manutenção e evolução do sistema.

Entre os padrões criacionais analisados, destacaram-se o **Factory Method** e o **Builder**. O Factory Method mostrou-se adequado para a criação de diferentes métodos de pagamento, permitindo acrescentar novas formas de pagamento com menor impacto no código existente. O Builder, por sua vez, mostrou-se adequado para a construção do bilhete electrónico, permitindo organizar a criação de um objecto que possui várias informações.

O **Singleton** foi mantido como uma possibilidade condicionada à necessidade real da arquitectura, evitando a sua utilização apenas por obrigação. Da mesma forma, os padrões **Abstract Factory** e **Prototype** foram analisados, mas não foram considerados obrigatórios para a implementação actual.

A arquitectura proposta, baseada em **React no frontend, Node.js/Express no backend e MySQL na persistência dos dados**, permite separar a interface, as regras de negócio e o acesso aos dados. Essa organização cria uma base adequada para futuras extensões do sistema.

Os testes definidos permitem avaliar tanto o funcionamento individual dos componentes como a integração entre eles, além de verificar se os princípios SOLID e os padrões seleccionados foram correctamente aplicados.

Conclui-se, portanto, que **Design Patterns não devem ser utilizados simplesmente pela quantidade de padrões implementados**, mas escolhidos de acordo com os problemas concretos encontrados no desenvolvimento. Esta abordagem permitiu construir uma solução mais coerente, modular e preparada para futuras evoluções.

REFERÊNCIAS BIBLIOGRÁFICAS

GAMMA, E.; HELM, R.; JOHNSON, R.; VLISSIDES, J. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.

PRESSMAN, R. S.; MAXIM, B. R. *Software Engineering: A Practitioner's Approach*. McGraw-Hill.

SOMMERVILLE, I. *Software Engineering*. Pearson.

Manual de Padrões de Projectos — Soluções Reutilizáveis. Material de apoio utilizado na elaboração do projecto.